

TriQ-KEX

Algorithm Specification and Supporting Documentation

Submitters: Lan Luo, Chaofeng Guan, Yunlei Zhao, Hong Wang, Jianhua Hou, and Tong Yu.

Contact: Lan Luo, lorraine0416@163.com

Abstract

TriQ-KEX is a two-message authenticated key-exchange protocol obtained by instantiating the FSXY generic transformation [1] with the submitted TriQ-KEM construction. It uses TriQ-KEM as the IND-CCA2 secure static KEM and TriQ-wKEM as the IND-CPA secure ephemeral KEM, while retaining the corresponding key, ciphertext, and shared-secret formats. Under the FSXY assumptions, TriQ-KEX targets CK^+ security in the standard model, based on the component KEM security, the required min-entropy conditions, and the PRF and KDF assumptions stated in this document.

Index Terms Code-Based Cryptography, Key Exchange, Key Encapsulation Mechanism, Quasi-Cyclic Codes, FSXY Transform.

CONTENTS

I	Introduction	3
I-A	Protocol overview	3
I-B	Protocol features	3
I-C	Organization	3
II	Preliminaries	4
II-A	Notation	4
III	Specifications	4
III-A	Protocol Flow	5
III-B	TriQ-KEM Interface	5
III-C	TriQ-wKEM	6
III-D	Static Keys for a Party	7
III-E	TriQ-KEX First Message Generation	7
III-F	TriQ-KEX Second Message Generation	8
III-G	TriQ-KEX Final Shared-Secret Derivation	8
III-H	Design Rationale	9
IV	Parameter Sets and Sizes	9
IV-A	Underlying TriQ-KEM Parameters	9
IV-B	Key, Message, and Session-Key Sizes	10
V	Security Statement	10
V-A	CK ⁺ Security and FSXY Reduction	11
V-B	Concrete Security for Submitted Parameters	11
VI	Algorithm Characteristics	12
VII	Implementation Notes and Known-Answer Tests	12
VII-A	Implementation Structure	12
VII-B	Implementation Performance	13
VII-C	Known-Answer Tests	14
VII-D	Intellectual Property Notice	14
	References	15

I Introduction

This document specifies TriQ-KEX, a two-message authenticated key-exchange protocol obtained by instantiating the Fujioka–Suzuki–Xagawa–Yoneyama (FSXY) generic transformation [1] with TriQ-KEM as the static KEM and TriQ-wKEM as the ephemeral KEM. The companion TriQ-KEM specification defines the underlying TriQ-PKE/TriQ-KEM algorithms, parameter derivation, QCS security estimates, decryption-failure analysis, and implementation benchmarks. This document records the TriQ-KEX protocol construction: static keys, the ephemeral KEM component, transmitted messages, transcript construction, session-key derivation, the FSXY security instantiation, KAT vectors, and implementation benchmarks for the submitted KEX instances.

I-A Protocol overview

At a high level, one TriQ-KEX session has the following three components.

- **Static keys:** Each party generates a static TriQ-KEM key pair and samples independent values $\sigma \in \mathcal{FS}$ and $\sigma' \in \{0, 1\}^\kappa$, used to derive the static KEM encapsulation randomness.
- **Two-message exchange:** The initiator sends a TriQ-KEM ciphertext under the responder static encapsulation key together with the ephemeral encapsulation key used by TriQ-wKEM; the responder returns a TriQ-KEM ciphertext under the initiator static encapsulation key and a TriQ-wKEM ciphertext formed with that ephemeral encapsulation key.
- **Session-key derivation:** The initiator and responder each obtain three KEM shared secrets (two from the static encapsulations and one from the ephemeral encapsulation) and derive the session key from these secrets together with the ordered protocol transcript using the FSXY session-key derivation rule.

I-B Protocol features

- **CK⁺ security:** The KEX security statement is CK⁺ security under the CK⁺ model and freshness definition used by FSXY, obtained by instantiating the FSXY theorem with the static IND-CCA2 TriQ-KEM component, the ephemeral IND-CPA TriQ-wKEM component, KEM shared secrets with sufficient min-entropy, and PRFs and a KDF satisfying the FSXY requirements.
- **Ephemeral KEM component:** The ephemeral TriQ-wKEM component uses the TriQ-KEM key-generation and encapsulation algorithms, but uses the weakened decapsulation rule required for the FSXY ephemeral KEM role.
- **Forward secrecy:** Under the CK⁺ freshness conditions and erasure of non-public per-session values after session completion, later disclosure of static secret keys alone does not compromise past session keys.

I-C Organization

The remainder of the document is organized as follows. Section II fixes notation and the FSXY interfaces used by the protocol. Section III gives the protocol flow, the KEM interfaces used by the protocol, the KEX algorithms, and the design rationale. Section IV lists the submitted parameter sets and the corresponding byte sizes. Section V states the CK⁺ security claim and records the FSXY reduction instantiation. Section VI summarizes the algorithm characteristics. Section VII gives implementation notes, KAT vectors, and reference/optimized benchmark measurements.

II Preliminaries

This section records the notation and the component definitions used by the TriQ-KEX protocol.

II-A Notation

- κ : the classical security parameter in bits (128, 256, 384, or 512 for the submitted instances).
- $\text{TriQ.KEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$: the IND-CCA2 secure TriQ-KEM component.
- $\text{TriQ.wKEM} = (\text{KeyGen}, \text{Encaps}, \text{Decaps}_w)$: the weakened TriQ encapsulation variant defined in Section III-C.
- $ek_i^{\text{KEM}}, dk_i^{\text{KEM}}$: the static TriQ-KEM encapsulation and decapsulation keys of party U_i .
- $pk_i^{\text{KEX}}, sk_i^{\text{KEX}}$: the KEX public and secret keys, with $pk_i^{\text{KEX}} := ek_i^{\text{KEM}}$.
- $ek_T^{\text{KEM}}, dk_T^{\text{KEM}}$: the ephemeral encapsulation and decapsulation keys used by TriQ-wKEM.
- $\mathcal{FS}$: the PRF-key space in the FSXY construction; for the submitted instances, $\mathcal{FS} = \{0, 1\}^\kappa$.
- $\mathcal{RS}_E$: the encapsulation-randomness space in the FSXY construction; the encapsulation randomness ρ is sampled from $\mathcal{RS}_E$.
- $\mathcal{RS}_G$: the key-generation-randomness space in the FSXY construction; the ephemeral key seed r_T is sampled from $\mathcal{RS}_G$.
- $F, F' : \{0, 1\}^* \times \mathcal{FS} \rightarrow \mathcal{RS}_E$: the two PRFs used in FSXY to compute the static KEM encapsulation randomness $\rho = F_\sigma(r) \oplus F'_{r'}(\sigma')$; $F_k(x)$ denotes $F(x, k)$, and similarly for F' .
- $G_{\text{FSXY}} : \mathcal{FS} \times \{0, 1\}^* \rightarrow \{0, 1\}^\kappa$: the PRF used for session-key derivation; $G_{\text{FSXY}, K}(X)$ denotes $G_{\text{FSXY}}(K, X)$.
- $\text{KDF} : \text{Salt} \times \mathcal{KS} \rightarrow \mathcal{FS}$: a key derivation function satisfying the FSXY KDF requirement for KEM shared secrets with sufficient min-entropy, where Salt is the KDF salt space, $\mathcal{KS}$ is the space of KEM shared secrets, $\mathcal{FS}$ is the PRF-key space, and $s \in \text{Salt}$ is a fixed non-secret public KDF salt included in the common public parameters. For the submitted instances, the KDF salt has κ bits.
- U_A, U_B : role labels for the initiator and responder.
- $x \in_R S$: x is sampled uniformly at random from the finite set S .
- ST : the session transcript defined in Section III-G.

The output key length of both KEM interfaces and the final session key match the classical security level κ in bits (16, 32, 48, or 64 bytes for the submitted instances). The KDF output is used as the key for the final PRF G_{FSXY} .

III Specifications

This section first gives the TriQ-KEX message flow, then defines the TriQ-KEM and TriQ-wKEM interfaces used by the protocol. The subsections starting with the static keys for a party specify the KEX algorithms shown in Figure 1, followed by design rationale.

III-A Protocol Flow

Figure 1 gives the complete TriQ-KEX two-message flow.

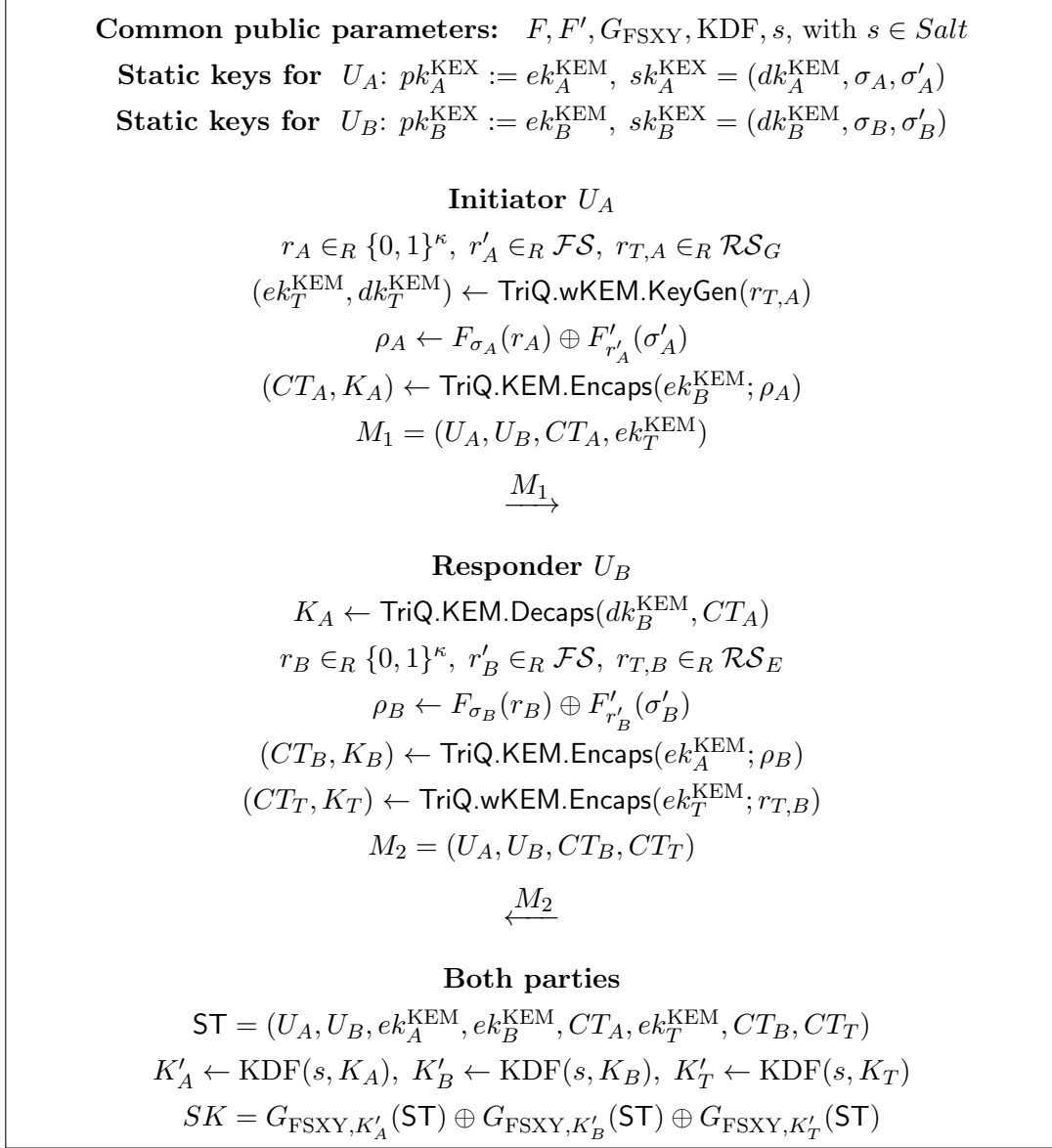


Figure 1: TriQ-KEX two-message protocol flow.

In Figure 1, K_A is the shared secret for the static TriQ-KEM ciphertext CT_A , K_B is the shared secret for the static TriQ-KEM ciphertext CT_B , and K_T is the shared secret for the ephemeral TriQ-wKEM ciphertext CT_T . Each value is output by the sender's encapsulation and by the receiver's corresponding decapsulation.

III-B TriQ-KEM Interface

For each submitted TriQ parameter set, TriQ-KEM is the FO-style KEM built from TriQ-PKE. The KEX protocol uses the following TriQ-KEM interface:

$$\begin{aligned} \text{TriQ.KEM.KeyGen}() &\rightarrow (ek^{\text{KEM}}, dk^{\text{KEM}}), \\ \text{TriQ.KEM.Encaps}(ek^{\text{KEM}}) &\rightarrow (c^{\text{KEM}}, K_{\text{enc}}), \\ \text{TriQ.KEM.Decaps}(dk^{\text{KEM}}, c^{\text{KEM}}) &\rightarrow K_{\text{dec}}. \end{aligned}$$

For TriQ.KEM, an argument $r \in \mathcal{RS}_G$ to KeyGen and a semicolon argument $\rho \in \mathcal{RS}_E$ to Encaps indicate fixed randomness only; the algorithms are unchanged. For the submitted TriQ instances, $\mathcal{RS}_G = \{0, 1\}^\kappa$; a value $r \in \mathcal{RS}_G$ is used as $seed^{\text{KEM}}$ in TriQ.KEM.KeyGen.

TriQ-KEM key generation samples $seed^{\text{KEM}}$, derives $seed^{\text{PKE}}$ and σ , and runs

$$(ek^{\text{PKE}}, dk^{\text{PKE}}) \leftarrow \text{TriQ.PKE.KeyGen}(seed^{\text{PKE}}).$$

It sets

$$ek^{\text{KEM}} = ek^{\text{PKE}}.$$

The decapsulation key is

$$dk^{\text{KEM}} = (ek^{\text{KEM}}, dk^{\text{PKE}}, \sigma, seed^{\text{KEM}}).$$

A TriQ-KEM ciphertext is parsed as

$$c^{\text{KEM}} = (c^{\text{PKE}}, salt).$$

For TriQ-KEM and TriQ-wKEM, a semicolon argument $\rho \in \mathcal{RS}_E$ fixes the internal randomness of Encaps; otherwise Encaps samples it internally. Here H_{KEM} , G_{KEM} , and J_{KEM} are the functions defined by the TriQ-KEM specification; the subscript distinguishes G_{KEM} from the FSXY session-key PRF G_{FSXY} . Encapsulation computes

$$(K, \theta) \leftarrow G_{\text{KEM}}(H_{\text{KEM}}(ek^{\text{KEM}}) \parallel m \parallel salt),$$

forms

$$c^{\text{PKE}} \leftarrow \text{TriQ.PKE.Encrypt}(ek^{\text{KEM}}, m, \theta),$$

and outputs $(c^{\text{KEM}}, K_{\text{enc}})$. Decapsulation reads ek^{KEM} , dk^{PKE} , and σ from dk^{KEM} , parses $c^{\text{KEM}} = (c^{\text{PKE}}, salt)$, and decrypts

$$m' \leftarrow \text{TriQ.PKE.Decrypt}(dk^{\text{PKE}}, c^{\text{PKE}}),$$

then recomputes

$$(K', \theta') \leftarrow G_{\text{KEM}}(H_{\text{KEM}}(ek^{\text{KEM}}) \parallel m' \parallel salt),$$

forms

$$c'^{\text{PKE}} \leftarrow \text{TriQ.PKE.Encrypt}(ek^{\text{KEM}}, m', \theta'), \quad c'^{\text{KEM}} = (c'^{\text{PKE}}, salt),$$

computes the rejection key from the input ciphertext,

$$\overline{K} \leftarrow J_{\text{KEM}}(H_{\text{KEM}}(ek^{\text{KEM}}) \parallel \sigma \parallel c^{\text{KEM}}).$$

It then compares c'^{KEM} with c^{KEM} . The output is K' if $c'^{\text{KEM}} = c^{\text{KEM}}$; otherwise the output is $\overline{K}$. TriQ-KEX uses this interface for the static KEM components.

III-C TriQ-wKEM

TriQ-wKEM uses the same parameter sets, key-generation algorithm, and encapsulation algorithm as TriQ-KEM. Therefore the ephemeral encapsulation key, decapsulation key, ciphertext, and shared secret are encoded as the corresponding TriQ-KEM outputs; TriQ-wKEM only changes decapsulation for the FSXY ephemeral role. Writing TriQ.wKEM.KeyGen(r) means that $r \in \mathcal{RS}_G$ fixes the key-generation randomness.

Let

$$c^{\text{KEM}} = (c^{\text{PKE}}, salt)$$

be the ciphertext input. The weakened decapsulation routine

$$\text{TriQ.wKEM.Decaps}_w(dk^{\text{KEM}}, c^{\text{KEM}})$$

proceeds as follows:

1. read ek^{KEM} and dk^{PKE} from dk^{KEM} ;

2. compute

$$m' \leftarrow \text{TriQ.PKE.Decrypt}(dk^{\text{PKE}}, c^{\text{PKE}}).$$

3. compute $(K', \theta') \leftarrow G_{\text{KEM}}(H_{\text{KEM}}(ek^{\text{KEM}}) \parallel m' \parallel \text{salt})$;

4. output K' .

No reencryption or comparison with the input ciphertext is performed, and no implicit-rejection value is used in Decaps_w .

III-D Static Keys for a Party

For a selected submitted TriQ parameter set, the static keys for a party U_i are generated as follows:

1. run $(ek_i^{\text{KEM}}, dk_i^{\text{KEM}}) \leftarrow \text{TriQ.KEM.KeyGen}()$ for the matching TriQ parameter set;

2. sample independent values $\sigma_i \in \mathcal{FS}$ and $\sigma'_i \in \{0, 1\}^\kappa$, where κ is the classical security level in bits (128, 256, 384, or 512 for the corresponding instance);

3. set

$$pk_i^{\text{KEX}} := ek_i^{\text{KEM}}, \quad sk_i^{\text{KEX}} = (dk_i^{\text{KEM}}, \sigma_i, \sigma'_i).$$

III-E TriQ-KEX First Message Generation

Let U_A be the initiator and U_B the responder. Given sk_A^{KEX} and $pk_B^{\text{KEX}} := ek_B^{\text{KEM}}$, the first message is generated as follows:

1. sample $r_A \in_R \{0, 1\}^\kappa$, $r'_A \in_R \mathcal{FS}$, and $r_{T,A} \in_R \mathcal{RS}_G$;

2. set

$$\rho_A = F_{\sigma_A}(r_A) \oplus F'_{\sigma'_A}(\sigma'_A),$$

where F and F' are the FSXY PRFs defined in Section II;

3. compute

$$(CT_A, K_A) \leftarrow \text{TriQ.KEM.Encaps}(ek_B^{\text{KEM}}; \rho_A);$$

4. run the ephemeral-key generation

$$(ek_T^{\text{KEM}}, dk_T^{\text{KEM}}) \leftarrow \text{TriQ.wKEM.KeyGen}(r_{T,A});$$

5. form

$$M_1 = (U_A, U_B, CT_A, ek_T^{\text{KEM}}).$$

The initiator keeps the local state

$$\text{state}_A = (CT_A, ek_T^{\text{KEM}}, dk_T^{\text{KEM}}, K_A)$$

for final shared-secret derivation after receiving the second message.

III-F TriQ-KEX Second Message Generation

Upon receiving

$$M_1 = (U_A, U_B, CT_A, ek_T^{\text{KEM}}),$$

the responder uses sk_B^{KEX} and $pk_A^{\text{KEX}} := ek_A^{\text{KEM}}$ to generate the second message:

1. compute

$$K_A \leftarrow \text{TriQ.KEM.Decaps}(dk_B^{\text{KEM}}, CT_A);$$

2. sample $r_B \in_R \{0, 1\}^\kappa$, $r'_B \in_R \mathcal{FS}$, and $r_{T,B} \in_R \mathcal{RS}_E$;

3. set

$$\rho_B = F_{\sigma_B}(r_B) \oplus F'_{r'_B}(\sigma'_B);$$

where F and F' are the FSXY PRFs defined in Section II;

4. compute

$$(CT_B, K_B) \leftarrow \text{TriQ.KEM.Encaps}(ek_A^{\text{KEM}}; \rho_B);$$

5. run

$$(CT_T, K_T) \leftarrow \text{TriQ.wKEM.Encaps}(ek_T^{\text{KEM}}; r_{T,B});$$

6. form

$$M_2 = (U_A, U_B, CT_B, CT_T).$$

The responder then holds the local state

$$\text{state}_B = (CT_B, CT_T, K_A, K_B, K_T).$$

III-G TriQ-KEX Final Shared-Secret Derivation

At the end of second-message generation, the responder has already obtained K_A , K_B , and K_T . After receiving $M_2 = (U_A, U_B, CT_B, CT_T)$, the initiator keeps the stored K_A and uses its static KEM decapsulation key and the ephemeral decapsulation key to compute

$$K_B \leftarrow \text{TriQ.KEM.Decaps}(dk_A^{\text{KEM}}, CT_B), \quad K_T \leftarrow \text{TriQ.wKEM.Decaps}_w(dk_T^{\text{KEM}}, CT_T).$$

Both parties then form the common session transcript

$$\text{ST} = (U_A, U_B, ek_A^{\text{KEM}}, ek_B^{\text{KEM}}, CT_A, ek_T^{\text{KEM}}, CT_B, CT_T),$$

derive the intermediate keys

$$K'_A \leftarrow \text{KDF}(s, K_A), \quad K'_B \leftarrow \text{KDF}(s, K_B), \quad K'_T \leftarrow \text{KDF}(s, K_T),$$

where s is the non-secret public KDF salt from the common public parameters. They output the session key

$$SK = G_{\text{FSXY}, K'_A}(\text{ST}) \oplus G_{\text{FSXY}, K'_B}(\text{ST}) \oplus G_{\text{FSXY}, K'_T}(\text{ST}),$$

where G_{FSXY} is a PRF whose output length matches the final session key length (κ bits, i.e., 16, 32, 48, or 64 bytes for the 128, 256, 384, and 512 instances respectively).

After outputting SK , each party erases its local state and the intermediate values used only for the final derivation.

III-H Design Rationale

This subsection records the design rationale for the submitted TriQ-KEX instantiation: the FSXY component choices, the PRF/KDF use, and correctness accounting.

FSXY component choices. The FSXY generic transformation [1] provides a theorem for obtaining a CK^+ secure two-message key-exchange construction from a static IND-CCA2 secure KEM, an ephemeral IND-CPA secure KEM, PRFs, and a KDF satisfying the stated FSXY requirements. TriQ-KEX instantiates this transformation as follows:

- static KEM encapsulations use the IND-CCA2 TriQ-KEM component;
- the ephemeral component is TriQ-wKEM, which uses the TriQ-KEM key-generation and encapsulation algorithms and the decapsulation rule Decaps_w defined in Section III-C.

The TriQ-PKE/TriQ-KEM parameter choices and component security analysis for these components are specified in the companion TriQ-KEM specification.

PRFs and KDF. The two FSXY PRFs F and F' determine the randomness passed to each static TriQ-KEM encapsulation. For either protocol direction,

$$\rho = F_\sigma(r) \oplus F'_{r'}(\sigma'),$$

where $\sigma \in \mathcal{FS}$ and $\sigma' \in \{0, 1\}^\kappa$ are part of the sender's static KEX secret key, while $r \in_R \{0, 1\}^\kappa$ and $r' \in_R \mathcal{FS}$ are sampled for that protocol run. The KDF is applied separately to K_A , K_B , and K_T with the public salt s , and the resulting values key the FSXY PRF G_{FSXY} on the ordered transcript, as specified in Section III-G. For KAT generation and benchmark execution, the submitted code realizes the FSXY PRF and KDF interfaces by domain-separated calls to the `API_PKC` auxiliary `pseudoXOF` function.

Correctness accounting. For honestly generated protocol messages, correctness of a completed TriQ-KEX session depends on two static TriQ-KEM decapsulations and one ephemeral TriQ-wKEM decapsulation. The component correctness-failure bounds are the natural-DFR bounds from the companion TriQ-KEM analysis and are summarized in Table 4. The session-level correctness loss is bounded by the sum of the two static TriQ-KEM component correctness-failure probabilities and the one ephemeral TriQ-wKEM component correctness-failure probability.

IV Parameter Sets and Sizes

This section records the underlying TriQ-KEM parameter values used by the submitted TriQ-KEX instances and the byte sizes of the TriQ-KEM encapsulation keys, decapsulation keys, and ciphertexts, together with the KEX keys, protocol messages, and session key. The parameter derivation and the code-based security analysis are specified in the companion TriQ-KEM specification and summarized in Section V.

IV-A Underlying TriQ-KEM Parameters

For each security level, TriQ-KEX uses the TriQ-KEM parameter values shown in Table 1. The table records the values needed in this specification to identify the submitted instances and compute the KEX byte sizes.

Here n is the quasi-cyclic ring length, n_1 is the outer RS length, n_2 is the inner repeated RM length, k is the PKE message length in bits, and w_x, w_y, w_r, w_e are the fixed-weight sampling

Table 1: Underlying TriQ-KEM parameter values for the submitted KEX instances

Instance	n	n_1	n_2	k	$w_{\mathbf{x}}, w_{\mathbf{y}}$	$w_{\mathbf{r}}$	$w_{\mathbf{e}}$
TriQ-KEX-128	16301	42	384	128	67	67	106
TriQ-KEX-256	50363	64	768	256	132	132	234
TriQ-KEX-384	97651	108	896	384	196	196	350
TriQ-KEX-512	157627	136	1152	512	260	260	475

parameters used by the underlying TriQ-PKE/TriQ-KEM component. The KEM seed length equals $\kappa/8$ bytes (16, 32, 48, or 64), and the TriQ-KEM ciphertext salt length is 32 bytes for all instances. Thus the key-generation randomness space used for `TriQ.KEM.KeyGen` and `TriQ.wKEM.KeyGen` in FSXY is $\mathcal{RS}_G = \{0, 1\}^\kappa$ for each submitted KEX instance.

IV-B Key, Message, and Session-Key Sizes

All sizes in this subsection are serialized byte sizes. Let $|ek^{\text{KEM}}|$, $|dk^{\text{KEM}}|$, and $|c^{\text{KEM}}|$ denote the byte sizes of the TriQ-KEM encapsulation key, decapsulation key, and ciphertext. For party U_i , the KEX public key returned by the submitted KEX interface is the encoded static TriQ-KEM encapsulation key:

$$\begin{aligned}
pk_i^{\text{KEX}} &:= ek_i^{\text{KEM}}, & |pk_i^{\text{KEX}}| &= |ek_i^{\text{KEM}}| = |ek^{\text{KEM}}|, \\
|sk_i^{\text{KEX}}| &= |dk_i^{\text{KEM}}| + 2 \cdot (\kappa/8) = |dk^{\text{KEM}}| + 2 \cdot (\kappa/8), \\
|M_1| &= |U_A| + |U_B| + |CT_A| + |ek_T^{\text{KEM}}|, \\
|M_2| &= |U_A| + |U_B| + |CT_B| + |CT_T|, \\
|SK| &= \kappa/8.
\end{aligned}$$

Here ek_i^{KEM} and dk_i^{KEM} are the static KEM encapsulation and decapsulation keys defined in Section III; the ephemeral encapsulation key and ciphertext use the same TriQ-KEM encodings, so $|ek_T^{\text{KEM}}| = |ek^{\text{KEM}}|$ and $|CT_A| = |CT_B| = |CT_T| = |c^{\text{KEM}}|$. In the submitted encoding, $|U_A| = |U_B| = 4$ bytes.

Table 2: Byte sizes of ek^{KEM} , dk^{KEM} , and c^{KEM}

Instance	$ ek^{\text{KEM}} $	$ dk^{\text{KEM}} $	$ c^{\text{KEM}} $
TriQ-KEX-128	2054	2102	4086
TriQ-KEX-256	6328	6424	12472
TriQ-KEX-384	12255	12399	24335
TriQ-KEX-512	19768	19960	39320

Table 3: Byte sizes of submitted TriQ-KEX keys, messages, and session key

Instance	$ pk_i^{\text{KEX}} $	$ sk_i^{\text{KEX}} $	$ M_1 $	$ M_2 $	$ SK $
TriQ-KEX-128	2054	2134	6148	8180	16
TriQ-KEX-256	6328	6488	18808	24952	32
TriQ-KEX-384	12255	12495	36598	48678	48
TriQ-KEX-512	19768	20088	59096	78648	64

V Security Statement

This section records the TriQ-KEX security claim and its instantiation of the FSXY security theorem.

V-A CK⁺ Security and FSXY Reduction

TriQ-KEX targets CK⁺ security under the CK⁺ model and freshness definition used by the FSXY generic transformation [1]. The FSXY theorem reduces this KEX security goal to the following component requirements:

- a static IND-CCA2 secure KEM (TriQ-KEM) with κ -min-entropy;
- an ephemeral IND-CPA secure weakened KEM (TriQ-wKEM) with κ -min-entropy;
- two independent PRFs F, F' for deriving the encapsulation randomness used by the static KEM encapsulations;
- a PRF G_{FSXY} for session key derivation;
- a KDF satisfying the FSXY KDF requirement for inputs with sufficient min-entropy.

Theorem V.1 (TriQ-KEX instantiation of FSXY Theorem 1 [1]). *Let κ be the security parameter. TriQ-KEX instantiates the FSXY construction with TriQ-KEM as the static IND-CCA2 κ -min-entropy KEM, TriQ-wKEM as the ephemeral IND-CPA κ -min-entropy KEM, and PRFs F, F', G_{FSXY} and a KDF satisfying the FSXY requirements. By FSXY Theorem 1, the TriQ-KEX construction specified in Section III is CK⁺ secure under the CK⁺ model and freshness definition used by FSXY.*

The proof is the generic FSXY game-hopping proof in Appendix A of [1], applied to the component choices above.

V-B Concrete Security for Submitted Parameters

The computational hardness inputs for TriQ-KEX are derived from the companion TriQ-KEM analysis of the underlying TriQ-PKE/TriQ-KEM construction. For the submitted instances, the relevant code-based assumptions are the 2-CW-QCSD problem associated with public-key recovery and the bounded-density-conditioned 3-CW-QCSD problem associated with ciphertext-randomness recovery. Both the static TriQ-KEM component and the ephemeral TriQ-wKEM component use these TriQ parameter sets; the KEX protocol does not introduce a separate code-based hardness assumption.

For the submitted parameter sets, Table 4 reports the numerical component estimates derived from the companion TriQ-KEM analysis. These estimates are used for the static TriQ-KEM component and the TriQ-wKEM component in the FSXY instantiation. The numerical inputs derived from the companion TriQ-KEM analysis are:

- the classical ISD and quantum-Prange bit-complexity estimates for the 2-CW-QCSD and bounded-density-conditioned 3-CW-QCSD problems;
- the natural DFR and post-rejection CDFR bounds for TriQ-PKE decryption in the corresponding honest-encapsulation and density-interval settings.

The “min classical ISD” column is the smallest classical ISD bit-complexity among the corresponding CW-QCSD estimates reported in the TriQ-KEM specification. The “min Q-Prange” column reports the smallest quantum-Prange bit-complexity estimate among the same component estimates. The CDFR column reports the post-rejection cumulative decryption-failure probability evaluated for the density-interval attack model under the prescribed attack budget. The natural DFR column is the single-query PKE decoding-failure probability under honest encapsulation and is the component value used for KEX correctness accounting.

The detailed CW-QCSD-based ISD analysis and decryption-failure derivation behind these component estimates are given in the companion TriQ-KEM specification.

Table 4: TriQ-KEM component estimates referenced by TriQ-KEX

Instance	min classical ISD (bits)	min Q-Prange (bits)	post-rejection CDFR	natural DFR
TriQ-KEX-128	146.67	95.18	$\leq 2^{-34.32}$	$\leq 2^{-121.90}$
TriQ-KEX-256	277.04	163.49	$\leq 2^{-85.63}$	$\leq 2^{-173.35}$
TriQ-KEX-384	404.13	229.43	$\leq 2^{-124.93}$	$\leq 2^{-209.57}$
TriQ-KEX-512	532.06	294.84	$\leq 2^{-146.56}$	$\leq 2^{-230.00}$

VI Algorithm Characteristics

TriQ-KEX is a code-based KEX protocol built from the submitted TriQ-KEM components by the FSXY generic transformation. Its main algorithmic characteristics are as follows.

- **Innovation.** TriQ-KEX provides a complete key-exchange instantiation for the TriQ family by combining the submitted TriQ-KEM component with the FSXY transformation. It gives a two-message KEX protocol whose security analysis is derived from the companion TriQ-KEM assumptions, decryption-failure bounds, and the FSXY session-key derivation argument.
- **Simplicity.** The protocol uses two transmitted messages, three KEM shared-secret values, and one final transcript-dependent session-key derivation. The same TriQ-KEM algorithms, encodings, and parameter sets are reused throughout the construction, keeping the KEX specification compact and the security argument traceable.
- **Flexibility.** The same protocol structure supports TriQ-KEX-128, TriQ-KEX-256, TriQ-KEX-384, and TriQ-KEX-512. The interface and byte encodings are consistent across the four submitted parameter sets and across the reference and optimized implementations.
- **Compatibility.** TriQ-KEX uses the submitted TriQ-PKE/TriQ-KEM parameter sets, API_PKC-style implementation interfaces, and the corresponding CW-QCSD-based security analysis. This aligns the KEX security categories with the companion TriQ-KEM instances in the same code-based design framework.
- **Scalability.** The KEX parameter sets scale with the companion TriQ-KEM categories, including KEM object sizes, message sizes, shared-secret lengths, and security estimates. This gives a uniform family of submitted KEX instances for the 128/256/384/512-bit classical security categories.
- **Implementation performance.** The submission includes both reference and optimized implementations. The measured costs for initialization, message generation, shared-secret derivation, and total KEX execution are reported in Section VII-B.

VII Implementation Notes and Known-Answer Tests

This section records implementation-level notes, benchmark measurements, and seed-driven known-answer test (KAT) vectors for TriQ-KEX.

VII-A Implementation Structure

Both the reference and optimized implementations instantiate the PKE/KEM operations with the submitted TriQ-KEM algorithms, parameter sets, and byte formats within the KEX code path. The reference implementation is portable ISO C. The optimized implementation targets mainstream x86_64 PC processors with AVX2-oriented routines for the underlying TriQ

computations. The submitted code does not link an external cryptographic library; for KAT generation and benchmark execution, hash and XOF support is provided by the `API_PKC` auxiliary functions. The submitted implementation represents the role labels U_A and U_B by the fixed four-byte strings `ID_A` and `ID_B`.

The performance tables report the following KEX API calls:

- **Init:** static KEX key generation for one party, reported as the average of the U_A and U_B calls;
- **M1:** first-message generation by U_A , consisting of one static TriQ-KEM encapsulation and one TriQ-wKEM key generation;
- **M2:** second-message generation by U_B , consisting of one static TriQ-KEM decapsulation, one static TriQ-KEM encapsulation, and one TriQ-wKEM encapsulation;
- **Derive U_A and Derive U_B :** session-key derivation by U_A and U_B , respectively;
- **Total:** one complete two-party run, including two **Init** calls.

For reproducibility, the submission package uses the following build configuration:

- compiler: GCC 15.2.0 from the MSYS2 UCRT64 environment;
- reference build flags: `-std=c99 -O3` plus warning flags;
- optimized build flags: `-std=c99 -O3` plus the AVX2/PCLMUL/BMI-family flags specified by the corresponding instance Makefile;
- the FSXY PRF and KDF interfaces are realized by domain-separated `pseudoXOF` calls from the `API_PKC` auxiliary interface for KAT generation and benchmark execution.

VII-B Implementation Performance

This subsection records measured CPU-cycle counts for the TriQ-KEX reference and optimized implementations of the four submitted instances.

Benchmarking methodology. Each benchmarked operation in Tables 5 and 6 is executed 10 000 times. Cycle counts are measured via `RDTSC` and reported as per-call average values. The benchmark platform is an Intel® Core™ i7-10700 CPU @ 2.90 GHz (8 cores, 16 threads) with 32 GB DDR4-2933 MHz RAM, running Microsoft Windows 11 Pro (24H2, 64-bit). The build environment is MSYS2 UCRT64 with GCC 15.2.0 (x86_64-w64-mingw32). The submitted implementations are written in C and compiled as C99 code. All implementations are compiled with `-std=c99 -O3`. The optimized implementations additionally use the AVX2/PCLMUL/BMI-family flags specified by the corresponding instance Makefile. The KEM hash/XOF calls, KEM deterministic-expansion calls, and FSXY PRF/KDF calls use the `API_PKC` auxiliary functions `sm3hash()`, `pseudohash()`, and `pseudoXOF()`. The benchmark uses the same deterministic harness as the submitted KAT generation.

Reference-implementation performance. Table 5 reports the average cycle counts for the TriQ-KEX reference implementation at each security level. The session-key length matches the classical security level ($\kappa/8$ bytes), and the benchmark uses the same `API_PKC` auxiliary functions and deterministic harness as the KAT generation.

Optimized-implementation performance. Table 6 reports the corresponding measurements for the optimized implementations. All four optimized instances use AVX2-oriented low-level arithmetic and vector routines for the underlying TriQ computations.

Table 5: TriQ-KEX reference-implementation runtime (average cycles, 10 000 repetitions)

Instance	Init	M1	M2	Derive U_A	Derive U_B	Total
TriQ-KEX-128	3 763 115	11 232 140	26 357 801	16 209 771	901 154	62 227 096
TriQ-KEX-256	21 813 895	65 384 921	152 638 202	90 218 329	2 752 191	354 621 433
TriQ-KEX-384	58 342 688	174 893 211	408 888 575	245 027 138	10 634 449	956 128 749
TriQ-KEX-512	106 556 756	318 732 147	743 254 282	442 221 116	17 209 747	1 734 530 804

Table 6: TriQ-KEX optimized-implementation runtime (average cycles, 10 000 repetitions)

Instance	Init	M1	M2	Derive U_A	Derive U_B	Total
TriQ-KEX-128	150 723	386 745	898 782	1 396 340	858 486	3 841 799
TriQ-KEX-256	514 108	1 417 886	3 358 406	4 712 345	2 641 741	13 158 594
TriQ-KEX-384	1 248 879	3 537 877	8 545 428	15 398 060	10 314 433	40 293 556
TriQ-KEX-512	3 402 719	9 231 296	20 812 634	28 328 655	16 523 547	81 701 570

VII-C Known-Answer Tests

Known-answer tests for TriQ-KEX follow the record format below. For each parameter set, every KAT record contains

Count, Seed_Len, Seed, Pass_Num,
 PKa_Len, PKa, SKa_Len, SKa, Init_Sta_Len, Init_Sta,
 PKb_Len, PKb, SKb_Len, SKb, Init_Stb_Len, Init_Stb,
 Pass1_Sta_Len, Pass1_Sta, M1_Len, M1,
 Pass2_Stb_Len, Pass2_Stb, M2_Len, M2, SS_Len, SS.

The submitted test vectors are in the top-level `Test_Vectors/` directory.

VII-D Intellectual Property Notice

The following patents may cover the submitted TriQ-KEX algorithm:

- US9094189 B2, *Cryptographic method for communicating confidential information*;
- FR 10/51190, *Procédé cryptographique de communication d'une information confidentielle*.

References

- [1] A. Fujioka, K. Suzuki, K. Xagawa, and K. Yoneyama, “Strongly secure authenticated key exchange from factoring, codes, and lattices,” in *Public-Key Cryptography – PKC 2012*, ser. Lecture Notes in Computer Science, 2012, extended version of the PKC 2012 paper.